

Marathon Training – Running Session Decision Support System

Development and Implementation of a Bayesian Network

Contents

<i>Introduction</i>	<i>2</i>
<i>Specification.....</i>	<i>2</i>
Purpose	2
Tasks	2
Objectives.....	3
Constraints	3
<i>Conceptual Design.....</i>	<i>3</i>
<i>Representation of Background Knowledge as a Bayesian Network.....</i>	<i>4</i>
Figure 1	4
Parent Nodes.....	4
Child Node.....	5
Estimating probabilities	5
Bayesian Network construction.....	5
<i>Analysis of questions</i>	<i>6</i>
Q1	6
Q2	6
Q3	6
<i>Conclusions and further work</i>	<i>7</i>
<i>References.....</i>	<i>8</i>
<i>Appendix.....</i>	<i>10</i>
Code.....	10
Questions	15
Probabilities	18

Introduction

Structuring marathon training can lead to more efficient training, less injuries, and better results. Using a system called periodisation a typical training block leading up to a race will be split into several periods or 'phases' each with a specific outcome. [1,2]

A typical training block would be split up into four phases, each of these phases have more emphasis on certain types of running session [3,4]:

Base - This phase is specifically for building endurance and will include mostly easy running and long runs with a few speed sessions.

Build – This builds on the base stage and starts to introduce more strength work so more speed sessions.

Peak – This stage gets the athlete into peak condition for their race so will include more long run sessions and speed work.

Taper – In this phase the athlete is winding down and recovering before their race so will consist of mostly easy runs and rest.

Specification

Purpose

In this project I will be designing and building a decision support system (DSS) to recommend the most suitable running sessions during an athletes training cycle for an upcoming race. The core of the DSS will consist of a Bayesian Network – a statistical model that takes random variables and uses a Directed Acyclic Graph (DAG) to model conditional probabilities [5]. The DSS will look at factors such as previous day's session, injury status, physical condition, and an athletes current training phase. The objectives are to optimise an athletes training by personalising session recommendations with the aim to maximise benefits such as performance and minimise injuries.

Tasks

The DSS will rely heavily on data collected to determine probabilities. Tasks of the DSS will include recording data such as injury history, previous training logs and subjective feedback on the athlete on their perceived exertion and current wellbeing. The DSS will then be used to estimate probabilities given previous evidence and provide decision logic to interpret the probabilistic reasoning and translate it into actionable advice – in this context, the session to run that day.

Objectives

The objectives of the DSS are to optimise and personalise an athletes training. A rigid training plan that ignores physical wellbeing, injury status and previous sessions efforts could lead to burnout and injuries. Having a system that can assess the athlete as they work through their training will maximise performance.

Constraints

The model relies on data. The availability of quality input data will be key. If this is poor or incomplete the recommendations will be inaccurate. The initial programming of the probabilities will also be a factor in accuracy and will need to be athlete specific in places, this could lead to inaccuracies if not done correctly. The Bayesian Network based DSS will also need to be updated in line with current sports science research.

Conceptual Design

The DSS will consist of three main areas. An input interface where the data can be collected and input into the system. The Bayesian Network – as the probabilistic reasoning engine and an output interface which will be used to output the session type to the user or coach.

The system could be integrated with existing fitness trackers and apps for automatic input of data via an API, this would also help keep data up to date by providing up to date data. The system could be integrated with training management systems for direct input by coaches. It could also be integrated with voice-controlled devices such as Amazon Echo which could provide a convenient input and output interface for the user.

Data for the system could be provided by either athlete or coach or a combination of both. It could have some information being directly fed by fitness trackers – such as previous session data. Certain data could be obtained by domain knowledge either through research or expert consultation. The DSS will be aimed at athletes and coaches as users. Stakeholders will also include sports scientists and researchers involved in system design.

The main output of the DSS will be recommendations for running session types. However, it could also be used to output the likelihoods of one of the parent node values given the child node value along with other parent node values. For example, the likelihood of the training phase the athlete is in given the running session type and injury status.

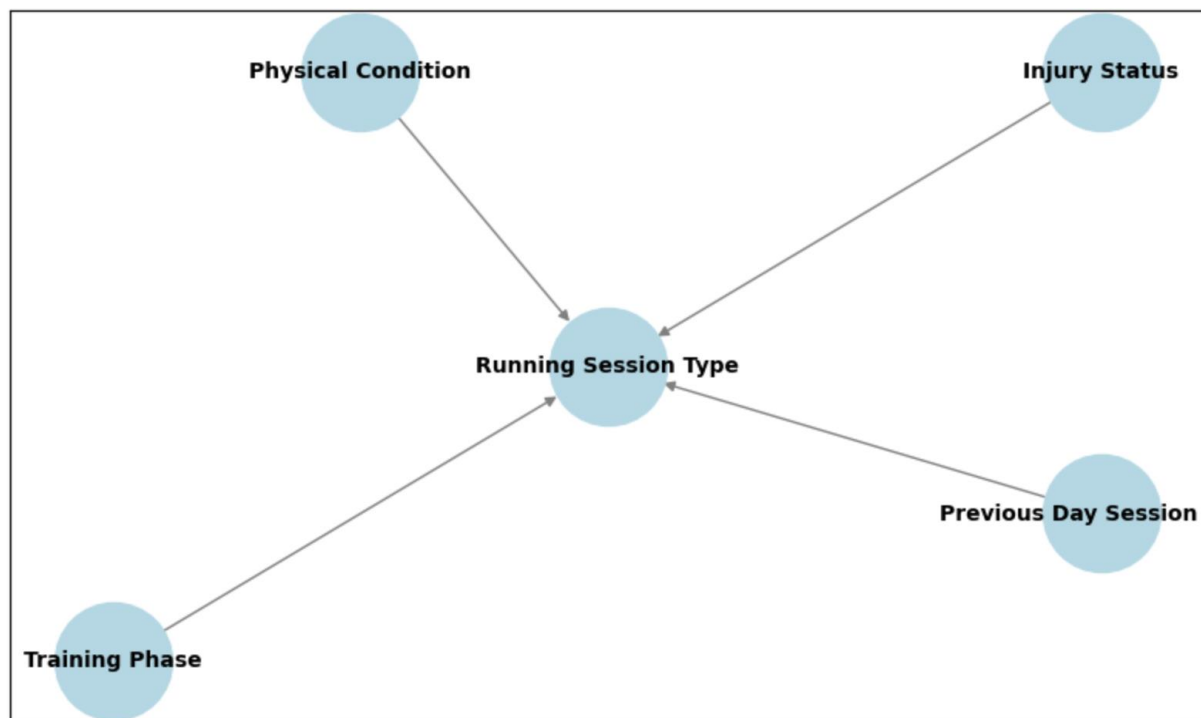
Although this system has many benefits that could maximise performance there are also risks associated. The system relies on correct input data which if not given could cause inaccurate advice which if acted on could lead to injuries.

Representation of Background Knowledge as a Bayesian Network

Although not exhaustive, the choice of nodes is based on personal experience as a long-time amateur marathon and ultra-distance endurance athlete but also on domain knowledge of sports science principles. There are many more factors, but the nodes chosen are I believe, and for simplicity, the key indicators here.

This type of Bayesian network is a 'common effects' network this is where multiple parent nodes influence a single child node. In this type of network, the parent nodes are typically conditionally independent. So, knowing the state of one parent node will not give information of another parent node unless you also have evidence of the child node [6,7]. The parent nodes in the network were selected as they have causal effects and directly influence and link to which session should be taken that day.

Figure 1



Parent Nodes

Previous session effort – Doing too many hard effort training sessions in succession can lead to injuries. However, too many rest days or easy days will not bring maximum benefits.

Injury status – Injured athletes will need more rest and athletes with minor injuries might not want to do avoid speed sessions and stick to easy runs to help with recovery.

Physical condition – you can be injury free but otherwise feel exhausted due to training or external factors, this should influence session type.

Training phase – as discussed above in different phases there is emphasis on certain session types.

All these nodes when considered together have an impact on the overall training regimen and will influence what session types should be considered next.

Child Node

Running Session Type – this will typically be the output of the DSS and will be based on conditional probabilities given from the input parent nodes.

Estimating probabilities

The probabilities input into the network were estimated using a mix of personal experience and domain knowledge, through talking to coaches at my local running club and through literature. For example, literature suggests a training plan should be around 40% base, 30% build, 20% peak and 10% taper [3]. It also suggests that recreational runners experience 7.7 injuries per 1000 hours of running [8]. This though is difficult to equate to the amount of time spent injured, with minor injuries and no injuries. So, for this metric I have used personal experience along with some discussions with professional coaches. For the other two parent nodes – previous day session and physical condition I again used myself as the main use case.

When it came to the probabilities for the child node – running session type I used a mixture of literature [9,10,11] and my own domain knowledge.

Bayesian Network construction

For the code in my project, I used the www.pgmpy.org website for examples and tutorials and some of the code used is based on examples from here [12,13,14]. To design the diagram of the network (Figure 1) I used NetworkX and based the code on examples and tutorials found [15,16]. See Appendix A for my detailed code.

To construct the Bayesian Network, I used python and the pgmpy library. This library is specifically made for working with Probabilistic Graphical Models such as Bayesian Networks.

Using pgmpy I first imported BayesianNetwork object which is used to construct the Bayesian Network and create a DAG, within this I defined the structure of the network which in this case connects each of the parent nodes to the child node.

I then used the TabularCPD object to create the Conditional Probability Distributions (CPDs). These represent the probability distributions for each of the nodes. For example, with Injury Status Variable Card represents how many different states there can be. The Values represent the probabilities, there is an 85% chance of having No Injury, 10% chance of having a Minor Injury, and 5% chance of being Injured. And the state names represent the states for the node.

The running session type child node is the most complex node as it brings in the parent nodes and shows how this is influenced by a combination of all these nodes. For this node, evidence and evidence_card is introduced. They represent that fact that this node is influenced by the parent nodes. As there were many combinations of probabilities for this node it got quite complex, and I decided to use excel to enter these. See Appendix B. I imported these into the notebook so they could be used.

Finally, all the CPDs were added to the network using the add.cpd method.

Analysis of questions

The next part of my code checks the network is behaving correctly by checking the CPDs, nodes and edges are valid. I also printed each CPD and the independencies so I could visually inspect them [17].

For the final part of the project, I used the VariableElimination [18] object to show the model can infer probabilities correctly and give value to the user. For this I proposed three questions. See appendix A 'Questions' section for more detail but here is a short summary:

Q1

When in the peak training phase, feeling good but you've recently had a hard session and have a minor injury what running session is recommended?

This question uses all nodes and would be a typical query to the network. It would be useful to gauge whether rest or an easy session would be the most appropriate.

Q2

When the athlete has no injuries, feels physically good and is in the build training phase. But previous session data is not available, what session is recommended?

This question tests the network using incomplete data. It shows the network is working and can give probabilities when only partial information is available. Which may sometimes be the case.

Q3

What is the most likely training phase an athlete is in given they have run a speed session and have a minor injury?

This uses the child node to predict the likelihood of one of the parent nodes (Training Phase). This is an interesting and insightful question for two reasons. Firstly, it is useful data for the coach to understand when in a training cycle an athlete is likely to

engage in such sessions when they have minor injuries. It would help in future coaching decisions when similar scenarios arise. Secondly, queries of this nature that use the child node help to validate the network.

Conclusions and further work

The Bayesian Network to support decisions on running session type is working as intended and could be used as a tool to tailor an athlete's session based on current and up to date data.

It would be interesting to develop the network further by adding more nodes such as mental wellbeing, type of injuries and previous performance data. This would produce better results but would have added complexity out of the scope of this project. Although this network was kept quite simple it was still challenging to assign the probabilities. I found adding the probabilities for the child CPD too difficult to manage in the python code so used excel to make it easier to construct and validate.

It would be interesting to develop the network further by adding feedback loops. For example, an athlete could provide feedback after the session such as perceived intensity or general satisfaction with the recommendation. This could be used to adjust the probabilities.

References

1. Williams TD, Toluoso DV, Fedewa MV, Esco MR. Comparison of periodized and non-periodized resistance training on maximal strength: A meta-analysis. *Sports Med* [Internet]. 2017 [cited 2024 Jan 2];47(10):2083–100. Available from: <https://pubmed.ncbi.nlm.nih.gov/28497285/>
2. Logue C. Periodization training: The ultimate guide [Internet]. *Marathon Handbook*. 2021 [cited 2024 Jan 2]. Available from: <https://marathonhandbook.com/periodization-training/>
3. Cavichioli L. Training periodization for long-distance runners: a guide to your new Personal Best [Internet]. *World's Marathons*. 2019 [cited 2024 Jan 2]. Available from: <https://worldsmarathons.com/article/training-periodization-for-long-distance-runners-a-guide-to-your-new-personal-best>
4. Murphy S. The science behind running your best marathon [Internet]. *Runner's World*. 2023 [cited 2024 Jan 2]. Available from: <https://www.runnersworld.com/uk/training/marathon/a46188782/science-marathon-running/>
5. Russell S, Norvig P, editors. Global Edition : A Modern Approach. In: *Artificial Intelligence: a Modern Approach*. p. 510–3.
6. Constantinou AC, Fenton N. Qmul.ac.uk. [cited 2024 Jan 2]. Available from: https://eecs.qmul.ac.uk/~norman/papers/Published_Significance_BayesianNetworks.pdf
7. Zhu J, Moore A. (aka Bayes nets, belief nets, directed graphical models) [Internet]. *Wisc.edu*. [cited 2024 Jan 2]. Available from: https://pages.cs.wisc.edu/~dyer/cs540/notes/13_bayes-net.pdf
8. Videbæk S, Bueno AM, Nielsen RO, Rasmussen S. Incidence of running-related injuries per 1000 h of running in different types of runners: A systematic review and meta-analysis. *Sports Med* [Internet]. 2015 [cited 2024 Jan 2];45(7):1017–26. Available from: <http://dx.doi.org/10.1007/s40279-015-0333-8>
9. Douglas S. How periodization can help you hit your training goals [Internet]. *Runner's World*. 2022 [cited 2024 Jan 2]. Available from: <https://www.runnersworld.com/advanced/a42218246/periodization-training-guide/>
10. Filipas L, Bonato M, Gallo G, Codella R. Effects of 16 weeks of pyramidal and polarized training intensity distributions in well-trained endurance runners. *Scand J Med Sci Sports* [Internet]. 2022 [cited 2024 Jan 2];32(3):498–511. Available from: <http://dx.doi.org/10.1111/sms.14101>
11. Casado A, González-Mohíno F, González-Ravé JM, Foster C. Training periodization, methods, intensity distribution, and volume in highly trained and

elite distance runners: A systematic review. *Int J Sports Physiol Perform* [Internet]. 2022 [cited 2024 Jan 2];17(6):820–33. Available from: <https://journals.humankinetics.com/view/journals/ijsp/17/6/article-p820.xml>

12 Supported Data Types — pgmpy 0.1.23 documentation [Internet]. Pgmpy.org. [cited 2024 Jan 2]. Available from: <http://www.pgmpy.org>

13 Tutorial Notebooks — pgmpy 0.1.23 documentation [Internet]. Pgmpy.org. [cited 2024 Jan 2]. Available from: <https://pgmpy.org/tutorial.html>

14 Example Notebooks — pgmpy 0.1.23 documentation [Internet]. Pgmpy.org. [cited 2024 Jan 2]. Available from: <https://pgmpy.org/examples.html>

15 Tutorial — NetworkX 3.2.1 documentation [Internet]. Networkx.org. [cited 2024 Jan 2]. Available from: <https://networkx.org/documentation/stable/tutorial.html>

16 Atta S. A tutorial on NetworkX: Network analysis in Python (part-I) [Internet]. The Startup. 2021 [cited 2024 Jan 2]. Available from: <https://medium.com/swlh/a-tutorial-on-networkx-network-analysis-in-python-part-i-43c1d35830b6>

17 Example Using the Earthquake network — pgmpy 0.1.23 documentation [Internet]. Pgmpy.org. [cited 2024 Jan 6]. Available from: <https://pgmpy.org/examples/Earthquake.html>

18 Variable Elimination — pgmpy 0.1.23 documentation [Internet]. Pgmpy.org. [cited 2024 Jan 2]. Available from: https://pgmpy.org/exact_infer/ve.html

Appendix

Code

Importing libraries

```
import pandas as pd

# visualisation
import matplotlib.pyplot as plt
import networkx as nx

# bayesian modelling
from pgmpy.models import BayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

Creating the network

Diagram of network

```
# code used below is based on examples and tutorials found here...
# https://networkx.org/documentation/stable/tutorial.html
# https://medium.com/swlh/a-tutorial-on-networkx-network-analysis-in-pytho
n-part-i-43c1d35830b6

# Define the structure of the Bayesian model
model = BayesianNetwork([
    ('Previous Day Session Effort', 'Running Session Type'),
    ('Injury Status', 'Running Session Type'),
    ('Physical Condition', 'Running Session Type'),
    ('Training Phase', 'Running Session Type')
])

# creates a directed graph from the model
nx_graph = nx.DiGraph()
nx_graph.add_edges_from(model.edges())

# sets the position for each node
pos = {
    'Running Session Type': (0.3, 0.5), # central
    'Previous Day Session Effort': (0.4, 0.45), # bottom right
    'Injury Status': (0.4, 0.6), # top right
    'Physical Condition': (0.25, 0.6), # top left
    'Training Phase': (0.2, 0.4) # bottom left
}

# sets the size of the figure
plt.figure(figsize=(10, 6))

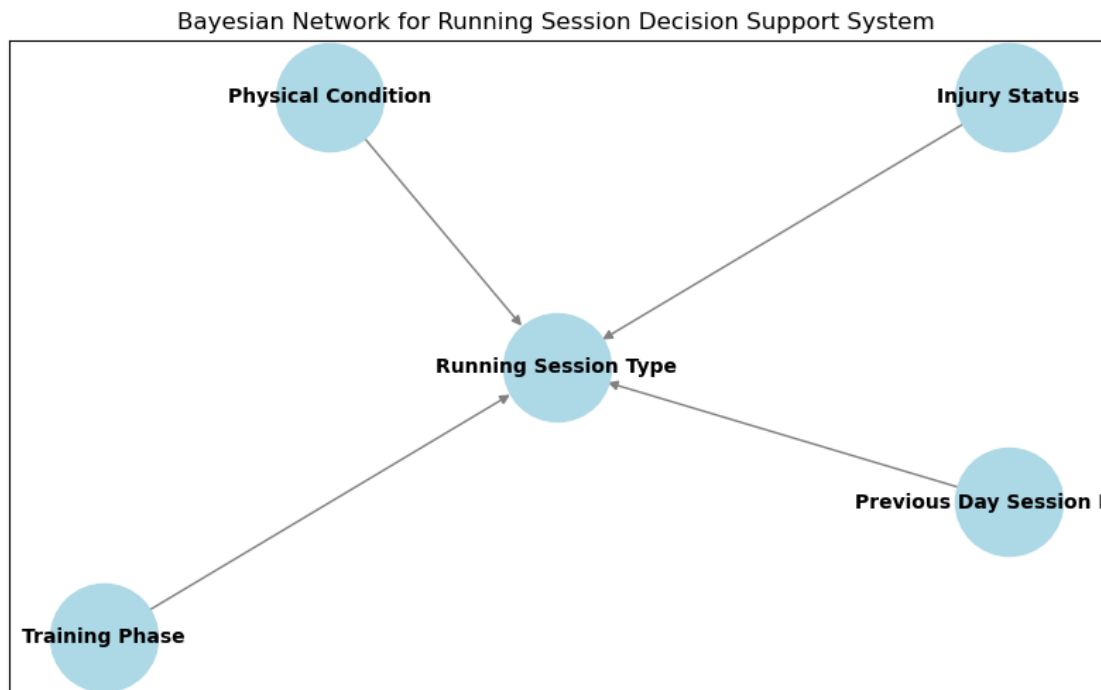
# draw the graph
nx.draw_networkx(nx_graph, pos, arrows=True, with_labels=True, node_color=
'lightblue',
```

```

edge_color='gray', node_size=3000, font_size=10, font_weight='bold')

# shows the plot
plt.title('Bayesian Network for Running Session Decision Support System')
plt.show()

```



Creating the bayesian network

```

# creating the network
rsr_model = BayesianNetwork([
    ('Previous Day Session Effort', 'Running Session Type'),
    ('Injury Status', 'Running Session Type'),
    ('Physical Condition', 'Running Session Type'),
    ('Training Phase', 'Running Session Type')
])

# cpd for previous session
cpd_prev_session = TabularCPD(variable='Previous Day Session Effort',
                               variable_card=3,
                               values=[[0.7],[0.2],[0.1]],
                               state_names={'Previous Day Session Effort':
['Easy', 'Hard', 'Rest']})

# cpd for injury status
cpd_injury_status = TabularCPD(variable='Injury Status',
                                variable_card=3,
                                values=[[0.85], [0.1], [0.05]],
                                state_names={'Injury Status': ['No Injury',
'Minor Injury', 'Injured']})

```

```

# cpd for physical condition
cpd_physical_condition = TabularCPD(variable='Physical Condition',
                                     variable_card=3,
                                     values=[[0.5],[0.3],[0.2]],
                                     state_names={'Physical Condition': ['Good', 'Neutral', 'Poor']})

# cpd for training phase
cpd_training_phase = TabularCPD(variable='Training Phase',
                                 variable_card=4,
                                 values=[[0.4],[0.3],[0.2],[0.1]],
                                 state_names={'Training Phase': ['Base', 'Build', 'Peak', 'Taper']})

# reading these probabilities into a df from a spreadsheet of values
run_session_type_values = pd.read_excel('run_session_type_values.xlsx')

# creating list from the data frame
long_run_vals = run_session_type_values['long_run_vals'].values.tolist()
easy_run_vals = run_session_type_values['easy_run_vals'].values.tolist()
speed_run_vals = run_session_type_values['speed_run_vals'].values.tolist()
rest_vals = run_session_type_values['rest_vals'].values.tolist()

# using lists from above to create the cpd
# cpd for run session type
cpd_run_type = TabularCPD(variable='Running Session Type', variable_card=4,
                           values=[long_run_vals, # probabilities for long
                                   run
                                   easy_run_vals, # probabilities for easy
                                   run
                                   speed_run_vals, # probabilities for spe
                                   ed run
                                   rest_vals],      # probabilities for res
                           state_names={'Running Session Type': ['Long', 'Easy', 'Speed', 'Rest'],
                                         'Previous Day Session Effort': ['Easy', 'Hard', 'Rest'],
                                         'Injury Status': ['No Injury', 'Minor Injury', 'Injured'],
                                         'Physical Condition': ['Good', 'Neutral', 'Poor'],
                                         'Training Phase': ['Base', 'Build', 'Peak', 'Taper']}},
                           evidence=['Previous Day Session Effort',
                                     'Injury Status',
                                     'Physical Condition'],

```

```

        'Training Phase'],
evidence_card=[3,3,3,4])

```

adding parameters to the model

```

rsr_model.add_cpds(
    cpd_prev_session, cpd_injury_status, cpd_physical_condition, cpd_train
    ing_phase, cpd_run_type
)

```

Validating the network model

I used some code from examples here to use for validation

<https://pgmpy.org/examples/Earthquake.html>

Checking cpd's

gets cpds input into model - checking they've all come through

```
rsr_model.get_cpds()
```

```

[<TabularCPD representing P(Previous Day Session Effort:3) at 0x7fbe25bdc9
10>,
 <TabularCPD representing P(Injury Status:3) at 0x7fbe25bdccd0>,
 <TabularCPD representing P(Physical Condition:3) at 0x7fbe25bdcca0>,
 <TabularCPD representing P(Training Phase:4) at 0x7fbe25bdcdc0>,
 <TabularCPD representing P(Running Session Type:4 | Previous Day Session
Effort:3, Injury Status:3, Physical Condition:3, Training Phase:4) at 0x7f
be26781e20>]

```

```
print(cpd_prev_session)
```

```

+-----+-----+
| Previous Day Session Effort(Easy) | 0.7 |
+-----+-----+
| Previous Day Session Effort(Hard) | 0.2 |
+-----+-----+
| Previous Day Session Effort(Rest) | 0.1 |
+-----+-----+

```

```
print(cpd_injury_status)
```

```

+-----+-----+
| Injury Status(No Injury) | 0.85 |
+-----+-----+
| Injury Status(Minor Injury) | 0.1 |
+-----+-----+
| Injury Status(Injured) | 0.05 |
+-----+-----+

```

```
print(cpd_physical_condition)
```

```

+-----+-----+
| Physical Condition(Good) | 0.5 |
+-----+-----+
| Physical Condition(Neutral) | 0.3 |
+-----+-----+

```

```

| Physical Condition(Poor)      | 0.2 |
+-----+-----+
print(cpd_training_phase)

+-----+-----+
| Training Phase(Base)         | 0.4 |
+-----+-----+
| Training Phase(Build)        | 0.3 |
+-----+-----+
| Training Phase(Peak)         | 0.2 |
+-----+-----+
| Training Phase(Taper)        | 0.1 |
+-----+-----+

# checks cpd's are valid
rsr_model.check_model()

True

# checking nodes
rsr_model.nodes()

NodeView(('Previous Day Session Effort', 'Running Session Type', 'Injury S
tatus', 'Physical Condition', 'Training Phase'))

# checking edges
rsr_model.edges()

OutEdgeView([('Previous Day Session Effort', 'Running Session Type'), ('In
jury Status', 'Running Session Type'), ('Physical Condition', 'Running Ses
sion Type'), ('Training Phase', 'Running Session Type')])

# checking independencies
rsr_model.get_independencies()

(Physical Condition  $\perp$  Previous Day Session Effort, Training Phase, Injury
Status)
(Physical Condition  $\perp$  Training Phase, Injury Status | Previous Day Session
Effort)
(Physical Condition  $\perp$  Previous Day Session Effort, Injury Status | Trainin
g Phase)
(Physical Condition  $\perp$  Previous Day Session Effort, Training Phase | Injury
Status)
(Physical Condition  $\perp$  Injury Status | Previous Day Session Effort, Trainin
g Phase)
(Physical Condition  $\perp$  Training Phase | Previous Day Session Effort, Injury
Status)
(Physical Condition  $\perp$  Previous Day Session Effort | Training Phase, Injury
Status)
(Previous Day Session Effort  $\perp$  Training Phase, Injury Status, Physical Con
dition)
(Previous Day Session Effort  $\perp$  Injury Status, Physical Condition | Trainin
g Phase)
(Previous Day Session Effort  $\perp$  Training Phase, Physical Condition | Injury

```

Status)
 (Previous Day Session Effort $\perp$ Training Phase, Injury Status | Physical Condition)
 (Previous Day Session Effort $\perp$ Physical Condition | Training Phase, Injury Status)
 (Previous Day Session Effort $\perp$ Injury Status | Training Phase, Physical Condition)
 (Previous Day Session Effort $\perp$ Training Phase | Injury Status, Physical Condition)
 (Training Phase $\perp$ Previous Day Session Effort, Injury Status, Physical Condition)
 (Training Phase $\perp$ Injury Status, Physical Condition | Previous Day Session Effort)
 (Training Phase $\perp$ Previous Day Session Effort, Physical Condition | Injury Status)
 (Training Phase $\perp$ Previous Day Session Effort, Injury Status | Physical Condition)
 (Training Phase $\perp$ Physical Condition | Previous Day Session Effort, Injury Status)
 (Training Phase $\perp$ Injury Status | Previous Day Session Effort, Physical Condition)
 (Training Phase $\perp$ Previous Day Session Effort | Injury Status, Physical Condition)
 (Injury Status $\perp$ Previous Day Session Effort, Training Phase, Physical Condition)
 (Injury Status $\perp$ Training Phase, Physical Condition | Previous Day Session Effort)
 (Injury Status $\perp$ Previous Day Session Effort, Physical Condition | Training Phase)
 (Injury Status $\perp$ Previous Day Session Effort, Training Phase | Physical Condition)
 (Injury Status $\perp$ Physical Condition | Previous Day Session Effort, Training Phase)
 (Injury Status $\perp$ Training Phase | Previous Day Session Effort, Physical Condition)
 (Injury Status $\perp$ Previous Day Session Effort | Training Phase, Physical Condition)

Questions

https://pgmpy.org/exact_infer/ve.html
 rsr_infer = VariableElimination(rsr_model)

Q1

An example using all of the nodes...

When in the peak training phase, feeling good but you've recently had a hard session and have a minor injury what running session is recommended?

Recommendation: Rest

Answer Explained:

The recommended session type is Rest, this makes sense as after a hard session with a slight injury, you would not want to risk further injury by running through the injury.

```
q1 = rsr_infer.query(variables=['Running Session Type'],
                      evidence={'Training Phase': 'Base',
                                'Physical Condition': 'Good',
                                'Injury Status': 'Minor Injury',
                                'Previous Day Session Effort': 'Hard'})
```

```
print(q1)
```

Running Session Type	phi(Running Session Type)
Running Session Type(Long)	0.2000
Running Session Type(Easy)	0.3500
Running Session Type(Speed)	0.0500
Running Session Type(Rest)	0.4000

Q2

Question uses partial information. Example of when not all information is available...

When the athlete has no injuries, feels physically good and is in the Build training phase. But previous session data is not available, what session is recommended?

Recommendation: Long

Answer Explained:

A long run is suggested, this is a sensible option as in the build phase lots of long and easy runs are required to build strength. When previous session data is not available. The best choice to give would be a long run. This is the most important session in this phase when there is no injury.

```
q2 = rsr_infer.query(variables=['Running Session Type'],
                      evidence={'Injury Status': 'No Injury',
                                'Physical Condition': 'Good',
                                'Training Phase': 'Build'})
```

```
print(q2)
```

Running Session Type	phi(Running Session Type)
Running Session Type(Long)	0.3700

Running Session Type(Easy)	0.3420
Running Session Type(Speed)	0.2100
Running Session Type(Rest)	0.0780

Q3

This uses the child node to predict the likelihood of one of the parent nodes...

What is the most likely training phase an athlete is in given they have run a speed session and have a minor injury?

Most likely phase: Peak

Answer Explained:

The most likely phase a runner is in given this information is Peak. In the peak stage an athlete will be giving it all to get into the best shape to perform in their race. So if they're doing a speed session and their injury status is Minor Injury Peak phase is the phase they are most likely to be in.

```
# Querying the probable states of 'Training Phase' given 'Running Session
Type' is
# 'Speed' and injury status is 'Minor Injury'
q3 = rsr_infer.query(variables=['Training Phase'],
                      evidence={'Running Session
Type': 'Speed',
                              'Injury Status':
'Minor Injury'})
print(q3)
```

Training Phase	phi(Training Phase)
Training Phase(Base)	0.2908
Training Phase(Build)	0.3178
Training Phase(Peak)	0.3498
Training Phase(Taper)	0.0415

Probabilities

Condition	long_run_vals	easy_run_vals	speed_run_vals	rest_vals
Easy, No Injury, Good,Base	0.45	0.35	0.15	0.05
Easy, No Injury, Good,Build	0.4	0.3	0.25	0.05
Easy, No Injury, Good,Peak	0.4	0.2	0.35	0.05
Easy, No Injury, Good,Taper	0.05	0.6	0.1	0.25
Easy, No Injury, Neutral,Base	0.4	0.4	0.15	0.05
Easy, No Injury, Neutral,Build	0.35	0.35	0.25	0.05
Easy, No Injury, Neutral,Peak	0.35	0.25	0.35	0.05
Easy, No Injury, Neutral,Taper	0.05	0.55	0.1	0.3
Easy, No Injury, Poor,Base	0.35	0.4	0.15	0.1
Easy, No Injury, Poor,Build	0.3	0.35	0.25	0.1
Easy, No Injury, Poor,Peak	0.3	0.25	0.35	0.1
Easy, No Injury, Poor,Taper	0.05	0.5	0.05	0.4
Easy, Minor Injury, Good,Base	0.3	0.4	0.1	0.2
Easy, Minor Injury, Good,Build	0.25	0.4	0.15	0.2
Easy, Minor Injury, Good,Peak	0.3	0.3	0.25	0.15
Easy, Minor Injury, Good,Taper	0.05	0.4	0.05	0.5
Easy, Minor Injury, Neutral,Base	0.25	0.45	0.1	0.2
Easy, Minor Injury, Neutral,Build	0.2	0.45	0.15	0.2
Easy, Minor Injury, Neutral,Peak	0.3	0.35	0.2	0.15
Easy, Minor Injury, Neutral,Taper	0.05	0.35	0.05	0.55
Easy, Minor Injury, Poor,Base	0.2	0.45	0.1	0.25
Easy, Minor Injury, Poor,Build	0.2	0.4	0.15	0.25
Easy, Minor Injury, Poor,Peak	0.25	0.35	0.2	0.2
Easy, Minor Injury, Poor,Taper	0.05	0.35	0.05	0.55
Easy, Injured, Good,Base	0.05	0.04	0.01	0.9
Easy, Injured, Good,Build	0.04	0.05	0.01	0.9
Easy, Injured, Good,Peak	0.06	0.04	0.02	0.88
Easy, Injured, Good,Taper	0.01	0.03	0.01	0.95
Easy, Injured, Neutral,Base	0.04	0.03	0.01	0.92
Easy, Injured, Neutral,Build	0.03	0.04	0.01	0.92
Easy, Injured, Neutral,Peak	0.05	0.03	0.02	0.9
Easy, Injured, Neutral,Taper	0.01	0.01	0.01	0.97
Easy, Injured, Poor,Base	0.03	0.02	0.01	0.94
Easy, Injured, Poor,Build	0.02	0.03	0.01	0.94
Easy, Injured, Poor,Peak	0.04	0.02	0.02	0.92
Easy, Injured, Poor,Taper	0.005	0.005	0.01	0.98
Hard, No Injury, Good,Base	0.3	0.45	0.05	0.2
Hard, No Injury, Good,Build	0.25	0.5	0.05	0.2
Hard, No Injury, Good,Peak	0.4	0.2	0.35	0.05
Hard, No Injury, Good,Taper	0.05	0.35	0.05	0.55
Hard, No Injury, Neutral,Base	0.25	0.4	0.05	0.3
Hard, No Injury, Neutral,Build	0.3	0.35	0.05	0.3
Hard, No Injury, Neutral,Peak	0.4	0.2	0.3	0.1

Hard, No Injury, Neutral,Taper	0.05	0.3	0.05	0.6
Hard, No Injury, Poor,Base	0.2	0.45	0.05	0.3
Hard, No Injury, Poor,Build	0.25	0.4	0.05	0.3
Hard, No Injury, Poor,Peak	0.35	0.25	0.25	0.15
Hard, No Injury, Poor,Taper	0.05	0.25	0.05	0.65
Hard, Minor Injury, Good,Base	0.2	0.35	0.05	0.4
Hard, Minor Injury, Good,Build	0.15	0.4	0.05	0.4
Hard, Minor Injury, Good,Peak	0.3	0.2	0.2	0.3
Hard, Minor Injury, Good,Taper	0.05	0.25	0.05	0.65
Hard, Minor Injury, Neutral,Base	0.2	0.3	0.05	0.45
Hard, Minor Injury, Neutral,Build	0.15	0.35	0.05	0.45
Hard, Minor Injury, Neutral,Peak	0.3	0.2	0.15	0.35
Hard, Minor Injury, Neutral,Taper	0.05	0.2	0.05	0.7
Hard, Minor Injury, Poor,Base	0.15	0.3	0.05	0.5
Hard, Minor Injury, Poor,Build	0.1	0.35	0.05	0.5
Hard, Minor Injury, Poor,Peak	0.25	0.2	0.1	0.45
Hard, Minor Injury, Poor,Taper	0.05	0.15	0.05	0.75
Hard, Injured, Good,Base	0.02	0.04	0.01	0.93
Hard, Injured, Good,Build	0.02	0.04	0.01	0.93
Hard, Injured, Good,Peak	0.03	0.04	0.01	0.92
Hard, Injured, Good,Taper	0.01	0.01	0.01	0.97
Hard, Injured, Neutral,Base	0.02	0.03	0.01	0.94
Hard, Injured, Neutral,Build	0.02	0.03	0.01	0.94
Hard, Injured, Neutral,Peak	0.02	0.04	0.01	0.93
Hard, Injured, Neutral,Taper	0.01	0.01	0.01	0.97
Hard, Injured, Poor,Base	0.01	0.02	0.01	0.96
Hard, Injured, Poor,Build	0.01	0.02	0.01	0.96
Hard, Injured, Poor,Peak	0.01	0.03	0.02	0.94
Hard, Injured, Poor,Taper	0.005	0.005	0.01	0.98
Rest, No Injury, Good,Base	0.45	0.37	0.15	0.03
Rest, No Injury, Good,Build	0.4	0.32	0.25	0.03
Rest, No Injury, Good,Peak	0.4	0.2	0.38	0.02
Rest, No Injury, Good,Taper	0.05	0.65	0.1	0.2
Rest, No Injury, Neutral,Base	0.4	0.41	0.15	0.04
Rest, No Injury, Neutral,Build	0.35	0.36	0.25	0.04
Rest, No Injury, Neutral,Peak	0.35	0.26	0.35	0.04
Rest, No Injury, Neutral,Taper	0.05	0.57	0.1	0.28
Rest, No Injury, Poor,Base	0.3	0.4	0.15	0.15
Rest, No Injury, Poor,Build	0.25	0.35	0.25	0.15
Rest, No Injury, Poor,Peak	0.25	0.25	0.35	0.15
Rest, No Injury, Poor,Taper	0.05	0.45	0.05	0.45
Rest, Minor Injury, Good,Base	0.25	0.55	0.05	0.15
Rest, Minor Injury, Good,Build	0.2	0.55	0.1	0.15
Rest, Minor Injury, Good,Peak	0.25	0.45	0.2	0.1
Rest, Minor Injury, Good,Taper	0.05	0.5	0.05	0.4
Rest, Minor Injury, Neutral,Base	0.25	0.5	0.1	0.15
Rest, Minor Injury, Neutral,Build	0.2	0.5	0.15	0.15
Rest, Minor Injury, Neutral,Peak	0.3	0.4	0.2	0.1
Rest, Minor Injury, Neutral,Taper	0.05	0.4	0.05	0.5
Rest, Minor Injury, Poor,Base	0.2	0.5	0.1	0.2
Rest, Minor Injury, Poor,Build	0.2	0.45	0.15	0.2

Rest, Minor Injury, Poor,Peak	0.25	0.4	0.2	0.15
Rest, Minor Injury, Poor,Taper	0.05	0.4	0.05	0.5
Rest, Injured, Good,Base	0.05	0.04	0.01	0.9
Rest, Injured, Good,Build	0.04	0.05	0.01	0.9
Rest, Injured, Good,Peak	0.06	0.05	0.01	0.88
Rest, Injured, Good,Taper	0.01	0.03	0.01	0.95
Rest, Injured, Neutral,Base	0.04	0.03	0.01	0.92
Rest, Injured, Neutral,Build	0.03	0.04	0.01	0.92
Rest, Injured, Neutral,Peak	0.05	0.03	0.02	0.9
Rest, Injured, Neutral,Taper	0.01	0.01	0.01	0.97
Rest, Injured, Poor,Base	0.03	0.02	0.01	0.94
Rest, Injured, Poor,Build	0.02	0.03	0.01	0.94
Rest, Injured, Poor,Peak	0.04	0.02	0.02	0.92
Rest, Injured, Poor,Taper	0.005	0.005	0.01	0.98